

Stack

- **Frontend:** Next.js + Tailwind. Mic button, campaign board, asset grid, change log
- **Backend:** FastAPI (Python). Agent orchestrator, request queue, validator
- **LLM:** agnes-3.0-flash via /v1/chat/completions with tool calling
- **Images:** agnes-image-2.5-flash via /v1/images/generations
- **Video:** agnes-video-2.5 Flash via /v1/videos, async and polled
- **Voice:** STT and TTS outside Agnes (Sarvam for Indic, or Whisper / browser Web Speech as fallback)
- **Overlay renderer:** HTML template → PNG (Playwright or Satori). It writes the text onto the generated images
- **Store:** SQLite (+ local /assets folder)

Pipeline

1. **Voice in** → STT → raw transcript (any language mix)
2. **Brief Agent** → extracts intent, tone, audiences, channels and offer details. Asks back by voice only if something critical is missing
3. **Offer Facts Lock** → a structured JSON record (item, discount, price, dates, timings, terms). The owner approves it and it's versioned. *Single source of truth.*
4. **Planner Agent** → campaign matrix: audiences × languages (EN/KN/HI) × channels (Instagram post, WhatsApp text, story, poster, 8 s reel)
5. **Copy Agent** → writes in each language natively from the intent, *not* by translating English, keeping placeholders like {discount} bound to the facts
6. **Creative Agent** → image prompt with no text → Agnes Image → overlay renderer stamps copy and facts onto it → final poster
7. **Video Agent** → queued 4–8 s clip with task-status UI and a labelled saved fallback
8. **Persona Panel** → Agnes role-plays 5 customer personas and scores each variant 1–10 on clarity, appeal, trust, local feel and call to action, with reasons
9. **Optimizer** → rewrites variants below threshold and re-scores (max 2 loops)
10. **Validator (deterministic, not an LLM)** → parses every output for numbers, dates and %, compares them to Offer Facts and blocks any mismatch. An LLM check for back-translation intent drift runs on top
11. **Board** → each asset tagged Approved / Pending / Changed / Blocked

Change handling

- A new voice command → **Diff Agent** classifies it as a fact change, tone change or scope change
- A fact change bumps the Offer Facts version and re-runs only the assets that use that field (dependency map: asset → facts used)
- Approved assets that aren't touched stay frozen
- The change log shows what changed, why and what's pending

Data model

- campaign(id, brand_voice, status)
- offer_facts(campaign_id, version, json, approved)
- asset(id, campaign_id, audience, lang, channel, type, content, facts_used[], facts_version, status, score)
- event_log(ts, actor, action, detail)

API routes

- `POST /voice` (audio → transcript → brief)
- `POST /facts/approve`
- `POST /campaign/generate`
- `POST /campaign/change` (voice or text)
- `GET /campaign/:id/board`
- `GET /jobs/:id` (video and image status)

Rate limits and safety

- One server-side queue with token buckets: text 10 RPM, image 10 RPM, video 1 RPM, with exponential backoff on 429
- API key only on the backend (`.env`)
- Cache prompts and outputs by hash so re-demos cost nothing
- Every saved fallback output is labelled "saved demo output"